

The Bernoulli Model: A Probabilistic Framework for Data Structures and Types

Bernoulli Data Type Project

May 9, 2025

Abstract

This paper introduces the Bernoulli Model, a probabilistic framework designed to handle uncertainty in data types, particularly Boolean values. The Bernoulli Model provides a formalism for optimizing space and accuracy in data representation through controlled trade-offs. We explore its applications in developing space-efficient data structures, such as Bloom filters and Count-Min sketches, and introduce the concept of Oblivious Data Types. The framework allows for $\mathcal{O}(1)$ time complexity while maintaining probabilistic guarantees on accuracy. This work forms a foundational part of a larger project aimed at enhancing data structure efficiency and error propagation analysis.

1 Introduction

The Bernoulli Model is a general framework for thinking about probabilistic data structures and types of a particular sort. A major motivation for developing the Bernoulli Model formalism is to use it as a foundation for creating Oblivious Data Types. The basic idea is that Bernoulli approximations have desirable properties for developing oblivious data types, and the formalism allows for reasoning about their correctness while making them more space-efficient by trading accuracy for space, all while maintaining $\mathcal{O}(1)$ time complexity.

Additionally, the Bernoulli Model provides a formalism for various probabilistic data structures, such as the Bloom filter, Count-Min sketch, and the Bernoulli data type family. These structures all derive from the Bernoulli Model and range from sets (like Bloom filters) to maps in a near-space optimal way, allowing for additional savings through controlled accuracy-space trade-offs.

1.1 Bernoulli Boolean

The Boolean type, denoted by `Bool`, models the set $\{\text{True}, \text{False}\}$. This paper explores replacing `Bool` with a type $\widehat{\text{Bool}}$, which represents a sort

of *noisy* Boolean. In general, we can have a Bernoulli type for any type T , denoted by $\widehat{T}$. As a special case, data structures like Bloom filters can be thought of as a Bernoulli approximation for the set indicator function, $1_A : X \mapsto \text{Bool}$. Indeed, we denote it by $1_{\widehat{A}} : X \mapsto \widehat{\text{Bool}}$. However, note that it's not *just* for the set-indicator function; it can be for any function in the computational basis of a type, e.g., the set-complement operator

$$c : 2^U \mapsto 2^U,$$

can introduce uncertainty in two different ways:

1. The set-complement function can be thought of as a Bernoulli approximation of the set-complement function, i.e., $\widehat{c} : 2^U \mapsto 2^{\widehat{U}}$. Here, an exact set comes in as input, and the output is a Bernoulli approximation of the input set.
2. The input set can be a Bernoulli approximation, i.e., $c : 2^{\widehat{U}} \mapsto 2^{\widehat{U}}$. Here, the input is a Bernoulli approximation of a set, and the output is also a Bernoulli approximation of the set-complement of the set. Here, the complement function is exact, but it propagates the uncertainty of the input set.

There are two types with fewer values than the Boolean type: the absurd type, denoted by void , which has no values (and thus values of this type cannot be constructed), and the unit type, denoted by $()$, which has only a single value, also denoted by $()$. Since there is only a single value of the unit type, there is no uncertainty in the unit type.

As degenerate cases, the Bernoulli Model of the absurd is just the absurd type, $\widehat{\text{void}} \equiv \text{void}$, and the Bernoulli Model of the unit type is just the unit type, $\widehat{()} \equiv ()$ since there is only one possible value. We may introduce uncertainty by composing such simple types, e.g., $() \oplus \text{Bool}$ has three possible values, $()$, True , and False . In this case, $\widehat{T}(\text{True})$ may represent any of the latent values True , False , or $()$, but most likely True . We see that the sum type can be Bernoulli, e.g., $\text{Bool} = \text{True} \oplus \text{False}$, and so $\widehat{\text{Bool}} = \widehat{\text{True}} \oplus \widehat{\text{False}}$. On the other hand, the product type $() \times \text{Bool}$ has two possible values, $() \times \text{True}$ and $() \times \text{False}$, and $\widehat{()} \times \widehat{\text{Bool}}$ has uncertainty only over the Booleans, i.e., $()$ is not uncertain. The product type is only Bernoulli if the types have more than one value.

The Boolean type Bool has two possible values, True and False , and is thus the first type for which we can introduce uncertainty.

2 Order of Bernoulli Types

Every Bernoulli Model also has an *order*, an integer greater than or equal to 0, which essentially describes the number of independent ways in which

the process that generated the Bernoulli approximation can produce errors. We denote that a Bernoulli Model of type T has order k with $\widehat{T}^{(k)}$. For the absurd and unit types, the maximum order is 0, and in general $T \equiv \widehat{T}^{(0)}$, i.e., if there are no ways to introduce errors, then the model is equivalent to the type itself.

Unless it is useful, we drop the order information and simply write $\widehat{T}$. In the Bernoulli Model, we may denote the latent value x that is being approximated using the notation $\widehat{T}(x)$. We say that x is latent if we do not know its value and are trying to approximate it using the Bernoulli Model approximation $\widehat{T}(x)$. In this case, the latent value x is unobservable, and we can think of $\widehat{T}(x)$ as a noisy measurement of x .

In the Bernoulli Boolean Model, $\widehat{\text{Bool}}(x)$ is a random Bernoulli variable such that

$$\Pr\{\widehat{\text{Bool}}(x) \neq x\} = \varepsilon(x) \quad (1)$$

for each $x \in \{0, 1\}$, where $0 < \varepsilon(x) < 1$ is the probability of an error. In most practical situations, $\varepsilon(x)$ is known or its expectation is known, and it can be pre-specified to balance factors like space complexity and accuracy.

3 Binary Channels

Let's begin by examining the Binary Symmetric Channel and the Binary Asymmetric Channel. The Bernoulli Boolean model can exhibit two distinct behaviors, represented as different "channels" through which Boolean values are transmitted:

1. **Binary Symmetric Channel (First-order Bernoulli Model):**
The probability of an equality error is the same for 1 and 0. We denote this by the type $\widehat{\text{Bool}}^{(1)}$.
2. **Binary Asymmetric Channel (Second-order Bernoulli Model):**
The probability of an equality error differs for 1 and 0. We denote this by the type $\widehat{\text{Bool}}^{(2)}$.

4 False Positives and Negatives

Errors in the Bernoulli Boolean model can be understood in terms of *false negatives* and *false positives*:

1. $\widehat{\text{False}} = \text{True}$ is a *false positive*.
2. $\widehat{\text{True}} = \text{False}$ is a *false negative*.

In the first-order model, the probability of a false negative equals the probability of a false positive. In the second-order model, these probabilities differ. In a specific but common version of the second-order Bernoulli Boolean model, false negatives occur with probability 0 and false positives occur with probability $0 < \varepsilon < 1$.

5 Prediction

$\hat{T}(x)$ is *correlated* with the latent value x , and thus provides information about the latent value. For instance, it allows one to predict x given $\hat{T}(x)$ better than if no observations were given whatsoever, assuming we know the error rate $\varepsilon(x)$ is better than a random guess, e.g., in the case of B_{Bool} , $\varepsilon(x) < 0.5$. If we have no prior information about the latent variable, the best (maximum likelihood) estimate of its value is the observation $\hat{T}(x)$.

However, because $\varepsilon(x)$ is normally known a priori, we can estimate the probability that the latent variable is $X = x$ using Bayes' rule:

$$\Pr\{X = x|\hat{x} = x\} \propto \Pr\{\hat{x} = x|X = x\} \Pr\{X = x\}, \quad (2)$$

where $\Pr\{X = x\}$ is the prior probability that $X = x$ and $\Pr\{\hat{x} = x|X = x\}$ is the probability that $\hat{x} = x$ given that $X = x$ (the probability that the observation is correct—the true positive rate τ or true negative rate ν).

In the first-order model, by Bayes law,

$$\Pr\{x|\hat{x}\} \propto \tau \Pr\{X = x\}, \quad (3)$$

where $\tau = 1 - \varepsilon$ is the true positive rate. The probability of observing x given that the latent value is x is $\Pr\{\hat{x} = x|X = x\} = \tau$. The probability of observing x given that the latent value is not x is $\Pr\{\hat{x} = x|X \neq x\} = \varepsilon$.

Assuming maximum ignorance (maximum entropy) about x (i.e., $X \sim \text{Bernoulli}(p = 0.5)$), the prediction's posterior above simplifies to $\Pr\{X = x|\hat{x} = x\} = \tau$ after we normalize the probabilities.

Suppose we have n noisy i.i.d. measurements of the latent variable x ,

$$\{\hat{x}, \dots, \hat{x}\}, \quad (4)$$

in which case the maximum likelihood estimate of x is the majority vote, i.e., the value that occurs most frequently in the observations.¹ As the number of independent sources of observations goes to infinity, the majority vote converges in probability to x ,

$$\lim_{n \rightarrow \infty} \Pr\{\text{majority}(\hat{x}_1, \dots, \hat{x}_n) = x\} = 1, \quad (5)$$

where $\hat{x}_i$ are independent Bernoulli variables (they do not need to be identically distributed). This is a consequence of the law of large numbers.

¹ Assuming the error rate is not larger than 0.5.

This is not a typical use-case for the Bernoulli Boolean model, since it will mostly be an analytical result of probabilistic data structures and algorithms that may be framed in the context of a Bernoulli Model.

6 Inducing Bernoulli Types

Here, we discuss how to generalize the results.

6.1 Unit Functions

We discussed the idea of the Bernoulli Model for **value types** like Bool. We can think of these Bernoulli Models as Bernoulli approximations of the set of unit types $() \mapsto X$. A unit type has no input, and it maps to a single constant value in the output. There are $|X|$ functions of this type.

If we replace $() \mapsto X$ with B_X or, equivalently, $B_{() \mapsto X}$, we allow for the possibility of errors. The confusion matrix for $B_{() \mapsto \text{Bool}}^{(2)}$ is provided in Table 1.

Table 1: Second-order Bernoulli Boolean Model for $() \mapsto \text{Bool}$

$x/B_{\text{Bool}}^{(2)}$	observe 1	observe 0
—rule latent 1	$\tau = 1 - \eta$	η
latent 0	ε	$\nu = 1 - \varepsilon$

So, we might observe $B_{() \mapsto \text{Bool}}(x) = 1$ and, according to the confusion matrix, the latent value x is 1 with probability τ (true positive rate) and 0 with probability ε (false positive rate). Likewise, we might observe $B_{() \mapsto \text{Bool}}(x) = 0$ and the latent value x is 1 with probability η (false negative rate) and 0 with probability ν (true negative rate).

We see that the Bernoulli Model for $() \mapsto \text{Bool}$ has a maximum order of 2, since η and ε are independent probabilities that fully describe the model. No additional free parameters are possible, since each row must sum to 1 (the total probability theorem), i.e., given a latent value x , the probability of observing 1 or 0 must sum to 1 since those are the only two possible outcomes in the Bernoulli Boolean Model.

We can think of this as the asymmetric binary channel, where the probability of an error is different for 1 and 0.

A first-order Bernoulli Boolean model is a model where $\epsilon = \varepsilon = \eta$. See the confusion matrix in Table 2.

We can see that there is only one free parameter, ϵ , corresponding to the first-order Bernoulli Boolean Model. We can think of this as a binary symmetric channel, where the probability of an error is the same for 1 and 0.

Table 2: First-Order Bernoulli Boolean Model for $() \mapsto \text{Bool}$

$x/B_{\text{Bool}}^{(1)}$	observe 1	observe 0
—rule latent 1	$\tau = 1 - \epsilon$	$\eta = \epsilon$
latent 0	$\varepsilon = \epsilon$	$\nu = 1 - \epsilon$

For completeness, we can write down the confusion matrix for the zeroth-order model, which is just the standard Boolean model, in Table 3.

Table 3: Zeroth-Order Bernoulli Boolean Model for $() \mapsto \text{Bool}$

$x/B_{\text{Bool}}^{(0)}$	observe 1	observe 0
—rule latent 1	1	0
latent 0	0	1

We see that there are 0 free parameters, and the model is deterministic.

6.2 Prediction: Boolean Values

Earlier, we discussed how to predict latent values from Bernoulli Model observations. Let's apply these insights to the first-order Bernoulli Model for $() \mapsto \text{Bool}$, which we can denote as $B_{\text{Bool}}^{(1)}$.

We can use Bayes' rule to compute the probability that the latent value is $X = 1$ given that we observed $B_{\text{Bool}}^{(1)} = 1$:

$$\Pr\{X = 1 | \hat{1}^{(1)} = 1\} = \frac{\Pr\{\hat{1}^{(1)} = 1 | X = 1\} \Pr\{X = 1\}}{\Pr\{\hat{1}^{(1)} = 1\}} \quad (6)$$

We know $\Pr\{\hat{1}^{(1)} = 1 | X = 1\}$ from the confusion matrix; it's just the true positive rate, $\tau = 1 - \epsilon$:

$$\Pr\{X = 1 | \hat{1}^{(1)} = 1\} = \frac{(1 - \epsilon) \Pr\{X = 1\}}{\Pr\{\hat{1}^{(1)} = 1\}} \quad (7)$$

What is $\Pr\{\hat{1}^{(1)} = 1\}$? By the total probability theorem:

$$\Pr\{\hat{1}^{(1)} = 1\} = \Pr\{\hat{1}^{(1)} = 1 | X = 1\} \Pr\{X = 1\} + \Pr\{\hat{1}^{(1)} = 1 | X = 0\} \Pr\{X = 0\}. \quad (8)$$

We can substitute in the confusion matrix values:

$$\Pr\{\hat{1}^{(1)} = 1\} = (1 - \epsilon) \Pr\{X = 1\} + \epsilon \Pr\{X = 0\}. \quad (9)$$

If we substitute this back into the expression for $\Pr\{X = 1|\hat{1}^{(1)} = 1\}$, divide the numerator and denominator by $\Pr\{X = 1\}$, and then simplify, we get

$$\Pr\{X = 1|\hat{1}^{(1)} = 1\} = \frac{1 - \epsilon}{1 - \epsilon(1 - q/(1 - q))} \quad (10)$$

where $q = \Pr\{X = 0\}$.

Let's evaluate $q = \Pr\{X = 0\}$ at some interesting points:

1. At $q = 0$, $\Pr\{X = 1|\hat{1}^{(1)} = 1\} = 1$. This makes sense. We know that the latent value 1 occurs with probability $1 - q = 1$, therefore whatever we observe, the latent value is 1.
2. As $q \rightarrow 1$, $\Pr\{X = 1|\hat{1}^{(1)} = 1\} \rightarrow 0$. This also makes sense; we know that the latent value 1 occurs with probability $q = 1$, so no matter what we observe, the latent value is 0.
3. At $q = 0.5$, $\Pr\{X = 1|\hat{1}^{(1)} = 1\} = 1 - \epsilon$. This is the maximum entropy case, where we have no prior information about the latent value and assume maximum ignorance.

7 Lifting Unary Functions

If we have a function $f : \text{Bool} \mapsto \text{Bool}$, then the space of all possible functions is given by Table 4.

Table 4: All possible functions of type $\text{Bool} \mapsto \text{Bool}$

f	$f(\text{True})$	$f(\text{False})$
—rule id	True	False
not	False	True
True	True	True
False	False	False

Suppose we replace the Boolean inputs with Bernoulli Boolean values and ask the question, "What is the probability that $f(\hat{x}^{(1)}) = f(x)$?"

Notice that $f(\hat{x}^{(1)})$ is $f(x)$ with some probability, but $f(x)$ may be latent depending on f . For the constant functions, True and False, we get the same function, i.e., $\text{True} : B_{\text{Bool}}^{(k)} \mapsto B_{\text{Bool}}^{(0)} \equiv \text{True} : \text{Bool} \mapsto \text{Bool}$ since $\text{True} : \text{Bool} \mapsto \text{Bool}$ always outputs True, and similarly for $\text{False} : \text{Bool} \mapsto \text{Bool}$.

However, the id and not functions are different. For instance, suppose $\Pr\{\hat{x}^{(1)} = x\} = p$. Then, when we input $B_{\text{Bool}}^{(1)}(\text{True})$ into id, we get the correct output True with probability p and the incorrect output False with probability $1 - p$. Likewise, when we input $B_{\text{Bool}}^{(1)}(\text{False})$ into id, we get the

correct output False with probability p and the incorrect output True with probability $1 - p$.

Since we can think of these outputs as either correct or incorrect with probability p and $1 - p$ respectively, they are Bernoulli Boolean values, e.g., this id function on Bernoulli Booleans is of type $B_{\text{Bool}}^{(1)} \mapsto B_{\text{Bool}}^{(1)}$. We monadically lift $\text{id} : \text{Bool} \mapsto \text{Bool}$ to a function of type $B_{\text{Bool}}^{(1)} \mapsto B_{\text{Bool}}^{(1)}$.

Notice that this is not a Bernoulli Model of $B_{\text{Bool} \mapsto \text{Bool}}$, but rather a function of type $B_{\text{Bool}} \mapsto B_{\text{Bool}}$. This may surprise the reader, but it is important to think about how these Bernoulli Models compose.

For instance, if we have the function $\text{True} : \text{Bool} \mapsto \text{Bool}$, then when we provide it with a Bernoulli Boolean value, we know that True is the correct output – there are no latent values. However, a Bernoulli Model of the (function) $\text{True} : \text{Bool} \mapsto \text{Bool}$ is a completely different concept. The Bernoulli approximation of this function is of type $B_{\text{Bool} \mapsto \text{Bool}}$, and the latent function, $\text{True} : \text{Bool} \mapsto \text{Bool}$, is not observable, but $B_{\text{Bool} \mapsto \text{Bool}}(\text{True})$ is observable and provides some information about the latent function.

Presumably, some process generated the Bernoulli approximation $B_{\text{Bool} \mapsto \text{Bool}}(\text{True})$, and we wish to use that approximate function as a replacement for the latent function we are actually interested in, which is $\text{True} : \text{Bool} \mapsto \text{Bool}$.

There are only 4 possible functions of type $\text{Bool} \mapsto \text{Bool}$, and in Table 5 we show the confusion matrix for the highest-order model, $B_{\text{Bool} \mapsto \text{Bool}}^{(12)}$.

Table 5: Bernoulli Model for $\text{Bool} \mapsto \text{Bool}$

	observe id	observe not	observe True	observe False
—rule latent id	p_{11}	p_{12}	p_{13}	p_{14}
latent not	p_{21}	p_{22}	p_{23}	p_{24}
latent True	p_{31}	p_{32}	p_{33}	p_{34}
latent False	p_{41}	p_{42}	p_{43}	p_{44}

Each row must sum to 1, $\sum_j p_{ij} = 1$, so we only have up to a maximum of $4(4 - 1) = 12$ degrees of freedom. This means the highest Bernoulli Boolean order is 12, but we normally drop the order information and just write $B_{\text{Bool} \mapsto \text{Bool}}$ (and propagate error rates using interval arithmetic).

Often, the order is either first-order for various reasons. The first-order Bernoulli Model of $\text{Bool} \mapsto \text{Bool}$ is a model where every entry in the confusion matrix is a function of some single value. The maximum entropy confusion matrix, given error rates ϵ , is given in Table 6.

When we have a Bernoulli Model approximation of some latent function of type $\text{Bool} \mapsto \text{Bool}$, we wish to store the error information in the output so that we can propagate error information through the computation.

We do this by saying that the output is a Bernoulli Boolean, because it may or may not be correct, i.e., the Bernoulli Model generates a function of type $\text{Bool} \mapsto B_{\text{Bool}}$ where the output is a Bernoulli Boolean value. In our

Table 6: Maximum Entropy Bernoulli Model for $\text{Bool} \mapsto \text{Bool}$

	observe id	observe not	observe True	observe False
—rule latent id	$1 - \epsilon$	$\epsilon/3$	$\epsilon/3$	$\epsilon/3$
latent not	$\epsilon/3$	$1 - \epsilon$	$\epsilon/3$	$\epsilon/3$
latent True	$\epsilon/3$	$\epsilon/3$	$1 - \epsilon$	$\epsilon/3$
latent False	$\epsilon/3$	$\epsilon/3$	$\epsilon/3$	$1 - \epsilon$

algorithms, we created a type system for this, using interval arithmetic to propagate the error rates through the computation.

Notice that the Bernoulli Model on $\text{Bool} \mapsto \text{Bool}$ does not change the type of the input, Bool . We can, of course, also provide as input to this function a Bernoulli Boolean, in which case we will usually get an even higher-order Bernoulli Boolean as output.

Since functions are values, we can ask the question, what is the probability that the latent function is equal to the observed function:

$$\Pr\{\widehat{\text{Bool} \mapsto \text{Bool}}(f) = f\}. \quad (11)$$

In this case, we are asking about the equality of the functions, which is mathematically equivalent to asking whether each input in the domain maps to the same output as the latent function:

$$\begin{aligned} &\Pr\{\widehat{\text{id}}(\text{True}) = \text{id}(\text{True})\} \times \\ &\Pr\{\widehat{\text{id}}(\text{False}) = \text{id}(\text{False})\} \end{aligned} \quad (12)$$

From the confusion matrix, we know this product of probabilities is $1 - \epsilon$. In fact, normally, the process that generates the Bernoulli Model of the latent function is based on these kinds of probabilities on individual inputs.

8 Higher-Order Bernoulli Models

If we are trying to estimate the latent function, a higher order complicates the estimation problem (more parameters to estimate). However, a higher-order may be *desirable* in many cases, since it allows for more capacity to approximate the latent function. In general, when looking at the confusion matrix, we want the diagonal to be as close to 1 as possible. For the off-diagonal elements, we want functions that are more similar to the latent function to have larger probabilities than functions that are less similar to the latent function. This is just a way of minimizing a loss function. This generally means that the higher-order Bernoulli Model may specify larger probabilities for the elements close to the diagonal and smaller probabilities for the elements further from the diagonal in a more flexible way. Of course, this is a trade-off between the number of parameters and the flexibility of

the model. In other cases, a higher-order Bernoulli Model may just be a natural result of a process or algorithm that generates samples from higher-order models, e.g., applying a first-order Bernoulli Map of $f : \text{Bool} \mapsto \text{Bool}$ where the approximation is most naturally defined over predicates on its application instead of, say, equality of the function itself.

It is very unlikely to be the case that the function is equal to the latent function, but it is very likely that the function is equal to the latent function on a large number of inputs.

We can think of the Bernoulli Map $\widehat{f}$ as a function that takes a Boolean value and returns a Bernoulli Boolean value, since applications may be wrong. Thus, we may think of the function as having the type

$$\text{Bool} \mapsto \widehat{\text{Bool}}.$$

Suppose the output type is a first-order Bernoulli Boolean $\widehat{\text{Bool}}1$ when given a Boolean value. Then, if we input a $\widehat{\text{Bool}}$ value, we get a higher-order $\widehat{\text{Bool}}$ as output, e.g., $\widehat{f}2 : \widehat{\text{Bool}}1 \mapsto \widehat{\text{Bool}}2$.

, and the input type is a Bernoulli Boolean $\text{Bool}^{\sigma(0)}$. The function f^σ maps from a Bernoulli Boolean $\text{Bool}^{\sigma(0)}$ to a Bernoulli Boolean $\widehat{\text{Bool}}^{(1)}$. If we apply this function to a Bernoulli Boolean value, we get a higher-order Bernoulli Boolean value, e.g., $f(\widehat{\text{True}})$ maps from a Bernoulli Boolean $\text{Bool}^{\sigma(0)}$ to a Bernoulli Boolean $\text{Bool}^{\sigma(1)}$.

9 Set-Indicator Functions

The set-indicator function is a function that maps an element of a set to a Boolean value. We can think of this as a function of type $X \mapsto \text{Bool}$, which returns true if the input is in the set and false otherwise.

Let us consider Bernoulli Models for set-indicator functions. The Bloom filter, for instance, is a Bernoulli model of the set-indicator function. It is a second-order model, since false negatives occur with probability 0 and false positives occur with probability ε . This is actually the *expectation* of the false positive rate, and the true false positive rate is a random variable that cannot usually be computed unless X is a finite set.

9.1 Approximate Set-Indicator Functions

Suppose we have a cryptographic hash function $h : X \mapsto \{0, 1\}^n$, where n is the number of bits in the hash value. We define an approximate set-indicator function, denoted by `hash_set`, in the following way:

- We are given a set A to (approximately) represent.
- We find a seed s such that $h(xs) \leq k$ for all $x \in A$.

- We do not look at any $x \notin A$, and by the properties of h , for $x \notin A$, $h(xs) \leq k$ with probability $k2^{-n}$ corresponding to a false positive, and otherwise a true negative occurs.
- We define membership as $x \in A \iff h(xs) \leq k$.

So, we construct a hash_set by finding a seed s such that $h(xs) \leq k$ and that is the only value we need to store, but since the probability that all $x \in A$ hash to a value less than k occurs with probability $k2^{-n}$, each trial is Bernoulli distributed with probability of success given by

$$\varepsilon(m, n, k) = \prod_{j=1}^m \frac{k+1}{2^n} \quad (13)$$

$$= \left(\frac{k+1}{2^n} \right)^m, \quad (14)$$

where $m = |A|$, k is the number of hash values we accept as positives, and n is the number of bits in the hash value.

The number of trials N is a geometric random variable with success probability $\varepsilon(m, n, k)$. The expected number of trials to find a seed s such that $h(xs) \leq k$ for all $x \in A$ is given by

$$E[N] = \frac{1}{\varepsilon(m, n, k)} = \left(\frac{2^n}{k+1} \right)^m. \quad (15)$$

Since the false positive rate $\varepsilon = (k+1)2^{-n}$, $n = \log(k/\varepsilon)$, we can reparameterize the space complexity as $\log(k/\varepsilon)$ bits per element. If we let $k = 0$, this simplifies to

$$\varepsilon = 2^{-n} \implies n = -\log \varepsilon$$

bits per element, which is the information-theoretic lower-bound, but we achieved it only by using an algorithm with exponential time complexity. We can do better by using different algorithms, but it comes at a cost to space complexity, such as a two-level hashing scheme, where the first level throws each element into a bucket, and the second level hashes the elements in the bucket. The number of buckets can be chosen as a trade-off between time and space complexity, e.g., if the number of buckets is b , then the expected number of elements in each bucket is m/b , and the expected number of trials to find a seed s such that $h(xs) \leq k$ for all $x \in A$ is given by

$$E[N] = \frac{b}{\varepsilon(m/b, n, k)} = \left(\frac{2^n}{k+1} \right)^{m/b}. \quad (16)$$

If we let $k = 0$, this simplifies to

$$E[N] = \frac{b}{\varepsilon(m/b, n, k)} = 2^{nm/b}.$$

This is the expected number of trials to find a seed s such that $h(xs) \leq k$ for all $x \in A$. How many bits, then, are we expected to need to store the seed s ? Assuming no fancy coding schemes (like a geometric code, which would be optimal), we can just store the seed in a fixed-size array whose size is the base-2 log of the seed,

$$\frac{nm}{b}$$

bits in total, or n/b bits per element (m elements in total).

Since $\varepsilon = 2^{-n}$, $n = -\log \varepsilon$, we can reparameterize the space complexity as

$$-\frac{\log \varepsilon}{b}$$

bits per element. This of course assumes that $m \gg b$, otherwise there may be a very uneven distribution of elements in the buckets, some with much larger than m/b elements.

9.2 Bernoulli Model for Set-Indicator Functions

The number of functions of type $X \mapsto \text{Bool}$ is $2^{|X|}$. We can think of these as the set of all possible set-indicator functions. We can approximate this set of functions with a Bernoulli Model, which we denote $B_{X \mapsto \text{Bool}}$.

We can use the simple `hash_set` construction to induce the Bernoulli Model for set-indicator functions. Specifically, in the `hash_set`, false positives occur with some positive probability ϵ and false negatives occur with probability 0.

Technically, however, the Bernoulli Model is being applied to the set-indicator function, but this also specifies a Bernoulli Model on the Boolean output of the set-indicator function:

$$\epsilon: X \times \mathcal{P}(X) \mapsto B_{\text{Bool}}. \quad (17)$$

Let's consider $X = \{1, 2\}$ and $A = \{2\}$. The confusion matrix for this construction is given in Table 7.

Table 7: Bernoulli Model for $X \mapsto \text{Bool}$				
—rule $1_\emptyset$	$1_\emptyset$ $(1 - \epsilon)^2$	$1_{\{1\}}$ $\epsilon(1 - \epsilon)$	$1_{\{2\}}$ $\epsilon(1 - \epsilon)$	$1_{\{1,2\}}$ ϵ^2
$1_{\{1\}}$	0	$1 - \epsilon$	0	ϵ
$1_{\{2\}}$	0	0	$1 - \epsilon$	ϵ
$1_{\{1,2\}}$	0	0	0	1

We do not know the latent set $A = \{2\}$, we only have the approximation $B_{X \mapsto \text{Bool}}(A)$, and we use this as a replacement for the latent set A .

Notice that if we start with $A = \{2\}$, in row 3, we have zeros in the first two columns for the empty set and $\{1\}$, which makes sense as by construction no false negatives are possible, only false positives.

We see that with probability $1 - \epsilon$, the output is correct, and with probability ϵ , the output is incorrect. This is a Bernoulli Model of the set-indicator function (or Bernoulli Set), and we can use this to provide information about the latent set.

On the equality of Bernoulli Sets, we can ask the question, what is the probability that $B_{X \mapsto \text{Bool}}(A) = A$? This is just $1 - \epsilon$, as shown in the confusion matrix. However, this is not typically what people are interested in. They are interested in the probability that an element that tests positive is in fact in the latent set A :

$$\Pr\{x \in B_{X \mapsto \text{Bool}}(A)\}. \quad (18)$$

This is just the true positive rate, which is $1 - \epsilon$ by construction. Thus, false positives occur with probability ϵ , and false negatives occur with probability 0. We see that when we do membership tests, we can think of the output as a Bernoulli Boolean value:

$$x \in B_{X \mapsto \text{Bool}}(A) : X \mapsto \text{Bool}^{(2)}. \quad (19)$$

10 Binary Logic Gates and Error Propagation

Let's consider the set of binary functions $f : (\text{Bool}, \text{Bool}) \rightarrow \text{Bool}$.

There are $2^2 = 4$ possible functions $f : \text{Bool} \rightarrow \text{Bool}$ since for each possible input, True or False, we have two possible outputs, True or False.

More generally, if we have $f : X \rightarrow Y$, then we have $|Y|^{|X|}$ possible functions, where $|\cdot|$ denotes the cardinality of a set. For instance, if $X = (\text{Bool}, \text{Bool})$ and $Y = \text{Bool}$, then we have $2^4 = 16$ possible functions, since $|X| = 4$ and $|Y| = 2$.

Each of these functions has a designated name, which we can use to refer to them, like and, xor, etc. Let's specifically look at and, which is defined in Table 8.

Table 8: Definition of and : $(\text{Bool}, \text{Bool}) \rightarrow \text{Bool}$

x_1	x_2	and(x_1, x_2)
True	True	True
True	False	False
False	True	False
False	False	False

Now, let's consider and : $(B_{\text{Bool}}^{(1)}, B_{\text{Bool}}^{(1)}) \rightarrow B_{\text{Bool}}^{(2)}$

This is more complicated than it might first seem. An error occurs if and returns True when it should return False, or vice versa. The input variables represent *latent* values, so they do not have a definite value.

We will go row by row, and examine the probability that the output is correct for each *output*.

10.1 Case 1: The Correct Output Is True

In order for the output to be true, both noisy inputs must be true, which is just the product of the probabilities of each condition being true since they are statistically independent outcomes.

10.2 Case 2: The Correct Output Is False Given $x_1 = \text{True}$ and $x_2 = \text{False}$

Consider $\text{and}(B_{\text{Bool}}^{(1)}(\text{True}), B_{\text{Bool}}^{(1)}(\text{False}))$. For this to be true, the first must be a true positive and the second must be a false positive, which is just $p_1 \cdot (1 - p_2)$. Since we are interested in the probability that it correctly maps to false, that is just $1 - p_1 \cdot (1 - p_2) = 1 - p_1 + p_1 \cdot p_2$.

10.3 Case 3: The Correct Output Is False Given $x_1 = \text{False}$ and $x_2 = \text{True}$

Consider $\text{and}(B_{\text{Bool}}^{(1)}(\text{False}), B_{\text{Bool}}^{(1)}(\text{True}))$. For this to be true, the first must be a false positive and the second must be a true positive, which is just $(1 - p_1) \cdot p_2$. Since we are interested in the probability that it maps correctly to false, that is just $1 - (1 - p_1) \cdot p_2 = 1 - p_2 + p_1 \cdot p_2$.

10.4 Case 4: The Correct Output Is False Given $x_1 = \text{False}$ and $x_2 = \text{False}$

Consider $\text{and}(B_{\text{Bool}}^{(1)}(\text{False}), B_{\text{Bool}}^{(1)}(\text{False}))$. For this to be true, both must be false positives, which is just $(1 - p_1) \cdot (1 - p_2)$. Since we are interested in the probability that it maps correctly to false, that is just $1 - (1 - p_1) \cdot (1 - p_2) = p_1 + p_2 - p_1 \cdot p_2$.

10.5 Summary

We can summarize the error propagation for the and function with Bernoulli inputs in Table 9.

We see that $\text{and} : (B_{\text{Bool}}^{(1)}, B_{\text{Bool}}^{(1)}) \rightarrow B_{\text{Bool}}^{(4)}$ induces an output that is a fourth-order Bernoulli Boolean. How is this possible when there are only two possible outputs? The answer is that the output is dependent on four different combinations of inputs.

Table 9: and with Bernoulli inputs

x_1	x_2	$\text{and}(x_1, x_2)$	$\text{Pr}\{\text{correct}\}$
—rule 1	1	1	$p_1 \cdot p_2$
1	0	0	$1 - p_1 + p_1 \cdot p_2$
0	1	0	$1 - p_2 + p_1 \cdot p_2$
0	0	0	$p_1 + p_2 - p_1 \cdot p_2$

Since x_1 and x_2 are *latent*, we can only talk about the probability that the output is correct or not. We see that when the output is 1, the probability that the output is correct is $p_1 \cdot p_2$. When the output is 0, the probability that it is correct is more complicated.

We could store all of this information in the type $B_{\text{Bool}}^{(4)}$, but it is probably more convenient to use interval arithmetic, where we store a range of probabilities for the probability that the Boolean value being stored is correct. The best choice is just the minimum length interval that contains all of the relevant probabilities for the output being correct. When the output is 1, we see that the minimum spanning interval is just $p_1 \cdot p_2$, and when the output is 0, the minimum spanning interval is the minimum span of

$$\min_span\{1 - p_1 + p_1 \cdot p_2, 1 - p_2 + p_1 \cdot p_2, p_1 + p_2 - p_1 \cdot p_2\} \quad (20)$$

As we compose more and more logic circuits together, we can keep track of the minimum spanning intervals on outputs using interval arithmetic.

11 Conclusion

The Bernoulli Model is a way of thinking about the uncertainty in the output of a function, and how that uncertainty propagates through a computation. Typically, the uncertainty is due to a trade-off between space complexity and accuracy. The more space we use to represent the function, the more closely it is expected to approximate the latent function.

This framework provides a formal foundation for developing space-efficient data structures with controllable error rates, such as Bloom filters and Count-Min sketches. It also enables the development of Oblivious Data Types, which maintain $\mathcal{O}(1)$ time complexity while making deliberate space-accuracy trade-offs.

By understanding how errors propagate through operations on Bernoulli types, we can build systems that maintain probabilistic guarantees throughout complex computations, enabling more efficient use of computational resources in scenarios where approximate answers are acceptable.

12 Bernoulli Maps

The Bernoulli Map provides a formal quantification of approximation error that is compatible with many algorithms, particularly data types that depend on hashing. This section extends our formalism to address how entire functions can be approximated using the Bernoulli model.

12.1 Latent Functions and Observable Approximations

Consider a *latent* function $p : X \rightarrow Y$ and an observable approximation or noisy version of p , denoted by $p^* : X \rightarrow Y$, such that $p^*(x) \neq p(x)$ with probability $e(x)$, where $\{e(x) : x \in X\}$ are a priori statistically independent random variables.

When we observe p^* , we know that it is a noisy model of some function $X \rightarrow Y$, but we do not know which function with certainty. If we are given no other prior information, then the maximum likelihood estimator of p is p^* itself.

12.2 Confusion Matrix for Function Spaces

Let the functions of type $X \rightarrow Y$ be named $p_1, p_2, \dots, p_n$, where n is the total number of functions in $X \rightarrow Y$. If no other information is given, $n = |Y|^{|X|}$.

For a simple example like $\text{Bool} \rightarrow \text{Bool}$ (where $n = 4$), the confusion matrix represents the probabilities of observing each function when the latent function is known:

Table 10: Confusion Matrix for Bernoulli Maps ($\text{Bool} \rightarrow \text{Bool}$)

latent / observe	p_1	p_2	p_3	p_4
—rule p_1	q_{11}	q_{12}	q_{13}	q_{14}
p_2	q_{21}	q_{22}	q_{23}	q_{24}
p_3	q_{31}	q_{32}	q_{33}	q_{34}
p_4	q_{41}	q_{42}	q_{43}	q_{44}

In this matrix, q_{ij} represents the probability that function p_i is observed as function p_j . Since each row must sum to 1, there are at most $n \cdot (n - 1)$ degrees of freedom, which in this case is $4 \cdot (4 - 1) = 12$.

This value corresponds to the *order* of the Bernoulli Model for $X \rightarrow Y$. In many practical applications, the order is 1, with most of the probability assigned to the diagonal elements, and the off-diagonal elements being nearly zero (or zero) but equal.

12.3 Notation and Type Signatures

We say that p^* is a *Bernoulli approximation* of p , and we denote this by writing $p^* \sim B_{X \rightarrow Y}(p)$, indicating that we observe p^* but know it is a random observation from a conditional distribution where we are conditioning on the latent function p to generate p^* .

If we observe a realization of $p^* \sim B_{X \rightarrow Y}(p)$, where p is latent, and we know it is a Bernoulli approximation, then p^* has the type $X \rightarrow B_Y$, representing a function that maps from X to Bernoulli approximations of elements in Y .

12.4 Information-Theoretic Properties

A key property of the Bernoulli Map confusion matrix is the entropy of the distribution $B_{X \rightarrow Y}$:

$$H(B_{X \rightarrow Y}) = - \sum_{i=1}^n \sum_{j=1}^n q_{ij} \log(q_{ij}) \quad (21)$$

We can also consider the conditional entropy distribution, $H(B_{X \rightarrow Y} | p_i)$, where p_i is the latent function:

$$H(B_{X \rightarrow Y} | p_i) = - \sum_{j=1}^n q_{ij} \log(q_{ij}) \quad (22)$$

In practice, we often know the confusion matrix a priori, or at least its properties, as the distortion results from a known process, such as a noisy channel, sensor, or measurement. This includes cases where a program introduces loss for compression, or from homomorphic encryption schemes where values are represented as products of trapdoors or one-way hashes.

12.5 Generating Bernoulli Approximations

A Bernoulli Map is a way of generating a Bernoulli approximation of a function. By the equivalence that data is code and code is data, any function can be represented as a data structure, and any data structure can be represented as a function. Therefore, we can model any data structure as a map, and then generate a Bernoulli approximation of that map.

For specific data structures, more efficient and specialized methods for generating Bernoulli approximations exist. Bloom filters are perhaps the most well-known example of this approach.

13 Applications of Bernoulli Maps

13.1 Set-Indicator Functions

The set-indicator function for a set A , denoted by $\mathbf{1}_A$, returns true if $x \in A$ and false otherwise. When we apply the Bernoulli Model to $\mathbf{1}_A$, we get a function of type $X \rightarrow B_{\text{Bool}}$. We observe $B_{X \rightarrow \text{Bool}}(\mathbf{1}_A)$ but do not know A with certainty.

Bloom filters and counting Bloom filters are common examples of this kind of approximation. The Bernoulli Map provides a unifying algorithm that can generate approximations of any computable functions, including set-indicator functions.

13.2 Primality Tests

Consider the primality test function $\text{isPrime} : \mathbb{Z} \rightarrow \text{Bool}$. While we know how to determine exactly whether an integer is prime (e.g., by checking divisibility by all smaller integers), the function is still *latent* in practice because the time required to compute it exactly for large inputs is prohibitive.

Instead, we can use randomized algorithms like the Miller-Rabin primality test, which can be conceptualized as a Bernoulli approximation $\text{isPrime}^* : \mathbb{Z} \rightarrow B_{\text{Bool}}$.

The Miller-Rabin test is based on Fermat's Little Theorem: if p is prime and a is any positive integer less than p , then $a^{p-1} \equiv 1 \pmod{p}$. The test works by randomly selecting values of a and checking whether the congruence holds. If it fails for any a , then p is definitely not prime. If it holds for all tested a , we say p is probably prime, with a specifiable false positive rate.

In essence, a particular seed value for the PRNG draws a sample function, a Bernoulli map, from $\text{isPrime}^* \sim B_{\mathbb{Z} \rightarrow \text{Bool}}(\text{isPrime})$.

13.3 Computational Basis

Given a set of functions $F = \{f_1, \dots, f_k\}$, we can define a Bernoulli model over F by generating realizations of $B_F(f_1), \dots, B_F(f_k)$, which we may denote as B_F .

For example, if we want to support both membership testing ($\in$) and equality testing ($=$) for sets, one approach is to generate a Bernoulli approximation for each operation. However, if we define $=$ in terms of $\in$, then that *induces* a Bernoulli model of $=$ through its dependence on $\in$.

13.4 Non-Regular Types

Bernoulli Models are not in general regular types, since a Bernoulli set A can have countably infinite representations, and it is generally impossible to

determine if two Bernoulli sets are the same.

This extends to the question of which latent set is being approximately modeled by a Bernoulli set. From this perspective, equality between a Bernoulli set and a regular set is not of type $(B_{\text{set}}, \text{set}) \rightarrow \text{Bool}$ but rather of type $(B_{\text{set}}, \text{set}) \rightarrow B_{\text{Bool}}$. In other words, we can only express the probability that a Bernoulli set represents a given latent set.

While this is typically less interesting than set-membership, it remains a valid question in the Bernoulli framework.

14 Error Propagation in Complex Data Types

As we consider more complex compound data types, which can be modeled as functions, we see that they can participate in the Bernoulli model in multiple ways. When a Bernoulli value is introduced into a computation, the entire computation produces a final result that is also a Bernoulli type, such as $B_{\text{pair}(T_1, T_2)}$ or $\text{pair}(T_1, B_{T_2})$.

This can be understood by considering a Universal Turing machine where programs are built by composing circuits of binary logic gates. If a single input to the circuit is replaced with a Bernoulli Boolean, the output becomes one or more Bernoulli Booleans. Similarly, if some logic gates are replaced with Bernoulli logic gates, the output is also a Bernoulli Boolean.

While we can always discard information about the uncertainty in the output, if the uncertainty is non-negligible, tracking it through the computation becomes valuable for understanding error propagation and making informed decisions about space-accuracy trade-offs.